Problem Set 3.

1)a) $K(x,z)$ = number of common words in x & z

$= |x \cap z|$

$\phi(x)$ $\phi(z)$

x = I like math
z = I like

} should get 2 words in common

$\phi(x) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ $\phi(z) = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

$\phi(x)^T \phi(z) = 2 = |x \cap z| = K(x,z)$

Yes, it is a kernel. Let $K(x,z)$ represent the inner products of the feature vectors where each document is mapped to a vector $\phi(x)$ in which each dimension corresponds to a unique word in the joint overall vocabulary of the two documents x and z. For each document, the feature vector has a 1 in that dimension if the word is present in the documents, and a 0 otherwise. The dot products of the vectors only contributes 1 if both documents have 1 in that dimension (word present in both $\rightarrow +1$) and 0 otherwise (since $1 \times 0 = 0$; i.e. word not present in 1 document $\rightarrow +0$). Overall, we'll get the number of unique words presented in both documents, or $|x \cap z|$.

1) b) $k(x,z) = x \cdot z$ is a kernel

- <u>scaling rule</u> $f(x)\, k_1(x,z)\, f(z)$

Let $f(x) = \frac{1}{\|x\|}$ $\quad f(z) = \frac{1}{\|z\|}$

$k_2(x,z) = f(x)\, k_1(x,z)\, f(z) = \frac{1}{\|x\|} \cdot x \cdot z \cdot \frac{1}{\|z\|} = \frac{x}{\|x\|} \cdot \frac{z}{\|z\|}$

$k_2(x,z)$ is a kernel $\quad k_3(x,z) = 1$ is trivial, valid kernel

- <u>sum rule</u> $k_3(x,z) + k_2(x,z)$ is a valid kernel

$k_4(x,z) = 1 + \frac{x}{\|x\|} \cdot \frac{z}{\|z\|}$ $\quad$ valid kernel ✓

- <u>product rule</u>

$k_5(x,z) = k_4(x,z) \cdot k_4(x,z) = \left(1 + \left(\frac{x}{\|x\|}\right)\left(\frac{z}{\|z\|}\right)\right)^2$ $\quad$ valid kernel ✓

$k_6(x,z) = k_5(x,z) \cdot k_4(x,z) = \left(1 + \left(\frac{x}{\|x\|}\right) \cdot \left(\frac{z}{\|z\|}\right)\right)^3$ $\quad$ valid kernel □

11) c) $K_\beta(x,z) = (1+\beta x\cdot z)^3$ $\beta>0$ $K(x,z) = (1+x\cdot z)^3$

Used online program to expand cubic (as allowed by problem)

$K_\beta(x,z) = 1 + 3\beta x_1 z_1 + 3\beta x_2 z_2 + 3\beta^2 x_1^2 z_1^2 + 3\beta^2 x_2^2 z_2^2 + \cancel{6\beta^2 x_1 x_2 z_1 z_2}$
$+ 6\beta^2 x_1 z_1 x_2 z_2 + \beta^3 x_1^3 z_1^3 + 3\beta^3 x_1 z_1 x_2^2 z_2^2 + \beta^3 x_2^3 z_2^3$
$+ 3\beta^3 x_1^2 z_1^2 x_2 z_2$

$$\begin{bmatrix} 1 \\ \sqrt{3\beta}\,x_1 \\ \sqrt{3\beta}\,x_2 \\ \sqrt{3\beta^2}\,x_1^2 \\ \sqrt{3\beta^2}\,x_2^2 \\ \sqrt{6\beta^2}\,x_1 x_2 \\ \sqrt{\beta^3}\,x_1^3 \\ \cancel{\sqrt{3\beta^3}\,x_1^2 x_2} \\ \cancel{\sqrt{3\beta^3}\,x_1 x_2^2} \\ \sqrt{\beta^3}\,x_2^3 \\ \sqrt{3\beta^3}\,x_1^2 x_2 \end{bmatrix}$$

$\star_1$
$\star_2$

$\star_1$ $\sqrt{\beta^3}\,x_2^3$

$\star_2$ $\sqrt{3\beta^3}\,x_1 x_2^2$

same coeff for z

$(1+x\cdot z)^3$

when $\beta=1$, we get the same feature map $\phi_\beta(\cdot)$, without the βs. This is a similarity. However, with $K(x,z)=(1+\beta x\cdot z)^3$, we have a scaling factor β, assuming $\beta\neq 1$.

The importance of the terms ~~returned~~ relative to each other changes.

$\star_1$ when $\beta>1$, $\beta^{3/2}$ is larger than β or $\sqrt{\beta}$, so the higher power terms have more weight or influence on the model. when $0<\beta<1$, $\beta^{3/2}$ is less than β or $\sqrt{\beta}$, so the higher power terms have less influence.

$\star_2$

2) a) $\frac{1}{2}\|\theta\|^2 = \frac{1}{2}(\theta_1^2 + \theta_2^2)$

$y_n \theta^T x_n \geq 1$ since we only have one vector, can say it is a support vector, or

$y_n \theta^T x_n = 1$ plug ~~$x = (a, e)^T$ and~~ $y = -1$ in

$(-1)\theta^T x_n = 1$

$\theta^T x_n = -1$ Theta must be some factor, since $\theta^T x_n = -1$ x scaled by

$\theta = c \cdot x$ $\quad x = \begin{bmatrix} a \\ e \end{bmatrix}$ $\quad \theta = \begin{bmatrix} c \cdot a \\ c \cdot e \end{bmatrix}$ *

$\theta^T x = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} [a \; e] = \theta_1 a + \theta_2 e = -1$

plug in scaled theta values (*)

$ca^2 + ce^2 = -1$

$c = \frac{-1}{a^2 + e^2}$

$\theta_1^* = \frac{-a}{a^2 + e^2}$ $\quad \theta_2^* = \frac{-e}{a^2 + e^2}$

$\theta^* = \begin{bmatrix} \frac{-a}{a^2+e^2} \\ \frac{-e}{a^2+e^2} \end{bmatrix}$

$$\frac{1}{2}\|\theta\|^2 = \frac{1}{2}(\theta_1^2 + \theta_2^2)$$

2) b) $y_n \theta^T x_n \geq 1$

$x_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $y_1 = 1$ $\quad$ $x_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ $y_2 = -1$

(1) $\theta^T x_n \geq 1$

(1) $\theta^T \begin{bmatrix} 1 \\ 1 \end{bmatrix} \geq 1$

$\theta_1 + \theta_2 \geq 1$

plug in $\rightarrow$

(-1) $\theta^T \begin{bmatrix} 1 \\ 0 \end{bmatrix} \geq 1$

$-(\theta_1 + \theta_2(0)) \geq 1$

$\theta_1 \leq -1$

minimizing, so $\theta_1 = -1$

$-1 + \theta_2 \geq 1$

$\theta_2 \geq 2$ again minimizing, so $\theta_2 = 2$

$$\theta^* = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

1 b/c SV

$$\gamma = \min \frac{|y_n \theta^T x_n|}{\|\theta^*\|} = \frac{1}{\|\theta^*\|} = \frac{1}{\sqrt{(-1)^2 + 2^2}} = \frac{1}{\sqrt{5}}$$

2) c) $y_i(\theta^T x + b) \geq 1$

$x_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (1) $[\theta_1\ \theta_2]\begin{bmatrix} 1 \\ 1 \end{bmatrix} + b = 1$

$y_1 = 1$

$\theta_1 + \theta_2 + b = 1$

$x_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ $(-1)[\theta_1\ \theta_2]\begin{bmatrix} 1 \\ 0 \end{bmatrix} + b = 1$

$y_1 = -1$ $(-1)(\theta_1 + b) = 1$

$\theta_1 + b = -1$

$$\begin{array}{r} \theta_1 + \theta_2 + b = 1 \\ -\ \theta_1 \quad\quad + b = -1 \\ \hline \theta_2 = 2 \end{array}$$

$\theta^* = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$

$+b = -1$

$\min \|\theta\|^2 = \theta_1^2 + \theta_2^2$

plug in θ_2

$\theta_1^2 + 4$

minimizing, so $\boxed{\theta_1 = 0}$

$0 + b = -1$

$b = -1$

$\gamma = \frac{y_i(\theta^T x_i + b)}{\|\theta^*\|} = \frac{1}{\sqrt{2^2+4}} = \frac{1}{2}$ (1 b/c SV)

$\frac{1}{2} > \frac{1}{\sqrt{5}}$ margin is bigger when we have an offset than when we do not

θ_1 is ~~smaller~~ larger (0 vs -1) but θ_2 is unchanged.
(smaller in magnitude)

CM146, Winter 2025
Problem Set 3: Deep learning, SVM, Kernels
Due February 28, 2025, 11:59pm PST

1 Kernels [8 pts]

(a) **(2 pts)** For any two documents $\boldsymbol{x}$ and $\boldsymbol{z}$, define $k(\boldsymbol{x}, \boldsymbol{z})$ to equal the number of unique words that occur in both $\boldsymbol{x}$ and $\boldsymbol{z}$ (i.e., the size of the intersection of the sets of words in the two documents). Is this function a kernel? Give justification for your answer.

Solution: On paper

(b) **(3 pts)** One way to construct kernels is to build them from simpler ones. We have seen various "construction rules", including the following: Assuming $k_1(\boldsymbol{x}, \boldsymbol{z})$ and $k_2(\boldsymbol{x}, \boldsymbol{z})$ are kernels, then so are

- (scaling) $f(\boldsymbol{x})k_1(\boldsymbol{x}, \boldsymbol{z})f(\boldsymbol{z})$ for any function $f(\boldsymbol{x}) \in \mathbb{R}$
- (sum) $k(\boldsymbol{x}, \boldsymbol{z}) = k_1(\boldsymbol{x}, \boldsymbol{z}) + k_2(\boldsymbol{x}, \boldsymbol{z})$
- (product) $k(\boldsymbol{x}, \boldsymbol{z}) = k_1(\boldsymbol{x}, \boldsymbol{z})k_2(\boldsymbol{x}, \boldsymbol{z})$

Using the above rules and the fact that $k(\boldsymbol{x}, \boldsymbol{z}) = \boldsymbol{x} \cdot \boldsymbol{z}$ is (clearly) a kernel, show that the following is also a kernel:

$$\left(1 + \left(\frac{\boldsymbol{x}}{||\boldsymbol{x}||}\right) \cdot \left(\frac{\boldsymbol{z}}{||\boldsymbol{z}||}\right)\right)^3$$

Solution: On paper

(c) **(3 pts)** Given vectors $\boldsymbol{x}$ and $\boldsymbol{z}$ in $\mathbb{R}^2$, define the kernel $k_\beta(\boldsymbol{x}, \boldsymbol{z}) = (1 + \beta \boldsymbol{x} \cdot \boldsymbol{z})^3$ for any value $\beta > 0$. Find the corresponding feature map $\phi_\beta(\cdot)$[1]. What are the similarities/differences from the kernel $k(\boldsymbol{x}, \boldsymbol{z}) = (1 + \boldsymbol{x} \cdot \boldsymbol{z})^3$, and what role does the parameter β play?

Solution: On paper

2 SVM [8 pts]

Suppose we are looking for a maximum-margin linear classifier *through the origin*, i.e. $b = 0$ (also hard margin, i.e., no slack variables). In other words, we minimize $\frac{1}{2}||\boldsymbol{\theta}||^2$ subject to $y_n \boldsymbol{\theta}^T \boldsymbol{x}_n \geq 1, n = 1, \ldots, N$.

(a) **(2 pts)** Given a single training vector $\boldsymbol{x} = (a, e)^T$ with label $y = -1$, what is the $\boldsymbol{\theta}^*$ that satisfies the above constrained minimization?

Solution: On paper

Parts of this assignment are adapted from course material by Tommi Jaakola (MIT), and Andrew Ng (Stanford), and Jenna Wiens (UMich).

[1] You may use any external program to expand the cubic.

(b) **(2 pts)** Suppose we have two training examples, $\boldsymbol{x}_1 = (1, 1)^T$ and $\boldsymbol{x}_2 = (1, 0)^T$ with labels $y_1 = 1$ and $y_2 = -1$. What is $\boldsymbol{\theta}^*$ in this case, and what is the margin γ?

Solution: On paper

(c) **(4 pts)** Suppose we now allow the offset parameter b to be non-zero. How would the classifier and the margin change in the previous question? What are $(\boldsymbol{\theta}^*, b^*)$ and γ? Compare your solutions with and without offset.

Solution: On paper

3 Implementation: Digit Recognizer [48 pts]

In this exercise, you will implement a digit recognizer in pytorch. Our data contains pairs of 28×28 images $\mathbf{x}_n$ and the corresponding digit labels $y_n \in \{0, 1, 2\}$. For simplicity, we view a 28×28 image $\mathbf{x}_n$ as a 784-dimensional vector by concatenating all the pixels together. In other words, $\mathbf{x}_n \in \mathbb{R}^{784}$. Your goal is to implement two digit recognizers (`OneLayerNetwork` and `TwoLayerNetwork`) and compare their performances.

---

code and data

- code : `CS146_Winter2025_PS3.ipynb`
- data : `ps3_train.csv`, `ps3_valid.csv`, `ps3_test.csv`

Please use your *@g.ucla.edu* email id to access the code and data. Similar to *PS1*, copy the colab notebook to your drive and make the changes. Mount the drive appropriately and copy the shared data folder to your drive to access via colab. The notebook has marked blocks where you need to code.

========= TODO : START =========

========= TODO : END ==========

Note: For parts (b)-(h), you are expected to copy-paste your code as a part of the solution in the submission pdf. Tip: If you are using LaTeX, check out the Minted package (example) for code highlighting.

Data Visualization and Preparation [10 pts]

i. **(2 pts)** Select three training examples with *different labels* and print out the images by using `plot_img` function. Include those images in your report.

Solution:

```
### part a: print out three training images with different labels
plot_img(X_train[0])
plot_img(X_train[1])
plot_img(X_train[7])
```

Figure 1: Part A

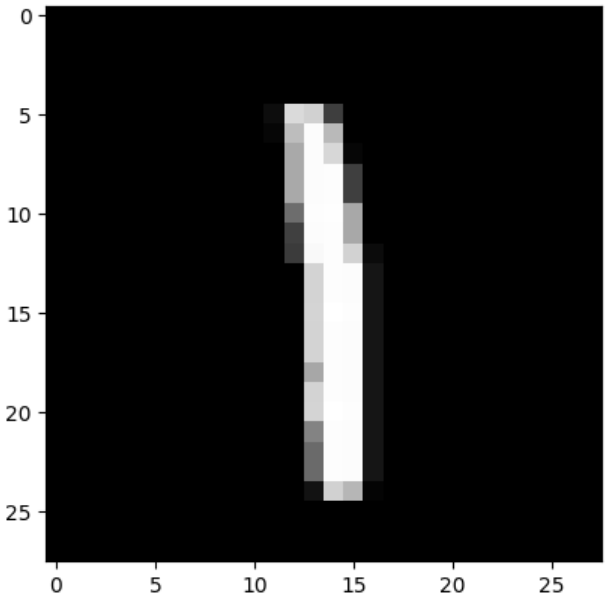


Figure 2: Part A, Example 1

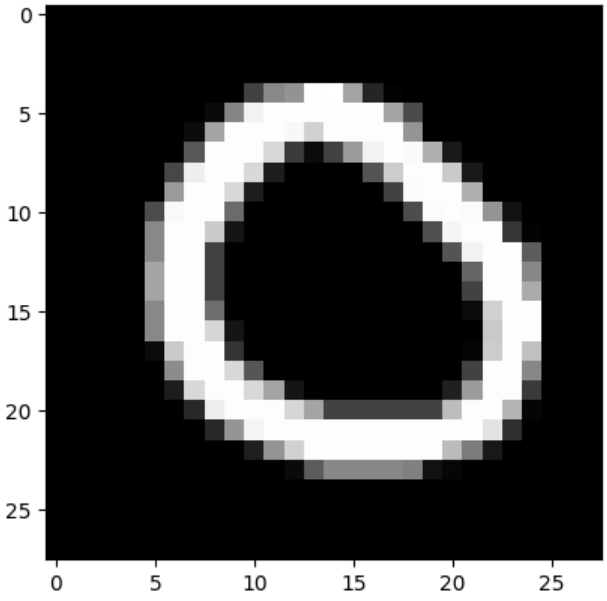


Figure 3: Part A, Example 2

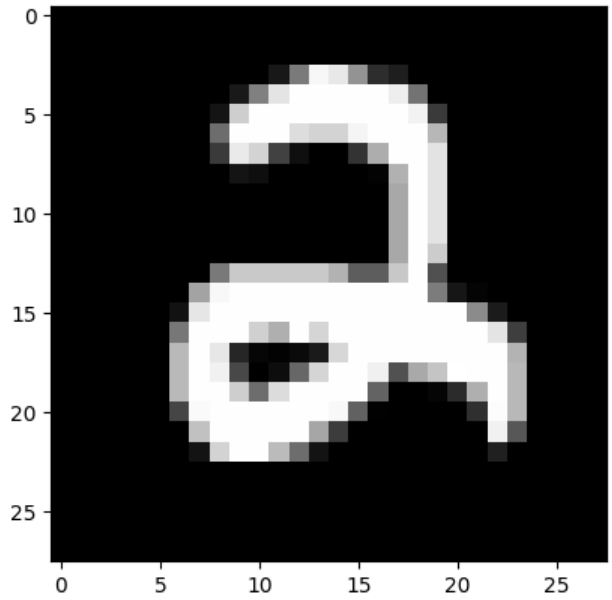


Figure 4: Part A, Example 3

ii. **(3 pts)** The loaded examples are numpy arrays. Convert the numpy arrays to PyTorch tensors.

Solution:

```
### part b: convert numpy arrays to tensors
X_train = torch.from_numpy(X_train)
y_train = torch.from_numpy(y_train)
X_valid = torch.from_numpy(X_valid)
y_valid = torch.from_numpy(y_valid)
X_test = torch.from_numpy(X_test)
y_test = torch.from_numpy(y_test)
```

Figure 5: Part B

iii. **(5 pts)** Prepare `train_loader`, `valid_loader`, and `test_loader` by using `TensorDataset` and `DataLoader`. We expect to get a batch of pairs $(\mathbf{x}_n, y_n)$ from the dataloader. Please set the batch size to 10 for all dataloaders and shuffle to True for `train_loader`. We need to set shuffle to True for `train_loader` so that we are training on randomly selected minibatches of data.

You can refer https://pytorch.org/docs/stable/data.html for more information about `TensorDataset` and `DataLoader`.

Solution:

```
### part c: prepare dataloaders for training, validation, and testing
###         we expect to get a batch of pairs (x_n, y_n) from the dataloader
train_loader = torch.utils.data.DataLoader(TensorDataset(X_train,y_train),
                                           batch_size=10,shuffle=True)
valid_loader = torch.utils.data.DataLoader(TensorDataset(X_valid,y_valid),
                                           batch_size=10)
test_loader = torch.utils.data.DataLoader(TensorDataset(X_test,y_test),
                                          batch_size=10)
```

Figure 6: Part C

One-Layer Network [15 pts]

For one-layer network, we consider a *784–3* network. In other words, we learn a 784×3 weight matrix $\mathbf{W}$. Given a $\mathbf{x}_n$, we can compute the probability vector $\mathbf{p}_n = \text{Softmax}(\mathbf{W}^\top \mathbf{x}_n)$, where $\mathbf{p}_{n,c}$ denotes the probability of class c. Then, we focus on the *cross entropy loss*

$$-\sum_{n=1}^{N}\sum_{c=1}^{C} \mathbf{1}(c = y_n) \log(\mathbf{p}_{n,c})$$

where N is the number of examples, C is the number of classes, and $\mathbf{1}$ is the indicator function.

iv. **(5 pts)** Implement the constructor of `OneLayerNetwork` with `torch.nn.Linear` and implement the `forward` function to compute the outputs of the single fully connected layer i.e. $\mathbf{W}^\top \mathbf{x}_n$. Notice that we do not compute the softmax function here since we will use `torch.nn.CrossEntropyLoss` later. The bias term is included in `torch.nn.Linear` by default, do *not* disable that option.

You can refer to https://pytorch.org/docs/stable/generated/torch.nn.Linear.html for more information about `torch.nn.Linear` and refer to https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html for more information about using `torch.nn.CrossEntropyLoss`.

Solution:

```
### part d: implement OneLayerNetwork with torch.nn.Linear
self.layer = torch.nn.Linear(784,3)
```

Figure 7: Part D, Part 2

```
### part d: implement the foward function
outputs = self.layer(x)
```

Figure 8: Part D, Part 2

v. **(2 pts)** Create an instance of `OneLayerNetwork`, set up a criterion with `torch.nn.CrossEntropyLoss`, and set up a SGD optimizer with learning rate 0.0005 by using `torch.optim.SGD`.

Solution:

```
### part e: prepare OneLayerNetwork, criterion, and optimizer
model_one = OneLayerNetwork()
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(params=model_one.parameters(), lr=0.0005)
```

Figure 9: Part E

You can refer to https://pytorch.org/docs/stable/optim.html for more information about `torch.optim.SGD`.

vi. **(8 pts)** Implement the training process. This includes forward pass, initializing gradients to zeros, computing loss, loss.backward, and updating model parameters. If you implement everything correctly, after running the `train` function in main, you should get results similar to the following.

```
Start training OneLayerNetwork...
| epoch  1 | train loss 1.075380 | train acc 0.453333 | valid loss ...
| epoch  2 | train loss 1.021195 | train acc 0.553333 | valid loss ...
| epoch  3 | train loss 0.972527 | train acc 0.626667 | valid loss ...
| epoch  4 | train loss 0.928296 | train acc 0.710000 | valid loss ...
...
```

Solution:

```
### part f: implement the training process
outputs = model.forward(batch_X)
optimizer.zero_grad()
loss = criterion(outputs, batch_y)
loss.backward()
optimizer.step()
```

Figure 10: Part F

```
Start training OneLayerNetwork...
| epoch  1 | train loss 1.075380 | train acc 0.453333 | valid loss 1.084895 |
| epoch  2 | train loss 1.021195 | train acc 0.553333 | valid loss 1.030888 |
| epoch  3 | train loss 0.972527 | train acc 0.626667 | valid loss 0.982578 |
| epoch  4 | train loss 0.928296 | train acc 0.710000 | valid loss 0.938814 |
| epoch  5 | train loss 0.887846 | train acc 0.783333 | valid loss 0.898887 |
```

Figure 11: Results, One Layer

Two-Layer Network [7 pts]

For two-layer network, we consider a *784–400–3* network. In other words, the first layer will consist of a fully connected layer with 784×400 weight matrix $\mathbf{W}_1$ and a second layer consisting of 400×3 weight matrix $\mathbf{W}_2$. Given a $\mathbf{x}_n$, we can compute the probability vector $\mathbf{p}_n = \text{Softmax}(\mathbf{W}_2^\top \sigma(\mathbf{W}_1^\top \mathbf{x}_n))$, where $\sigma(.)$ is the element-wise sigmoid function. Again, we focus on the *cross entropy loss*, hence the network will implement $\mathbf{W}_2^\top \sigma(\mathbf{W}_1^\top \mathbf{x}_n)$ (note the softmax will be taken care of implicitly in our loss). The bias term is included in `torch.nn.Linear` by default, do *not* disable that option.

vii. **(5 pts)** Implement the constructor of `TwoLayerNetwork` with `torch.nn.Linear` and implement the `forward` function to compute $\mathbf{W}_2^\top \sigma(\mathbf{W}_1^\top \mathbf{x}_n)$.

```
### part g: implement TwoLayerNetwork with torch.nn.Linear
self.layer1 = torch.nn.Linear(784,400)
self.layer2 = torch.nn.Linear(400,3)
```

Figure 12: Part G, Constructor

Solution:

```
    ### part g: implement the foward function
    fn = torch.nn.Sigmoid()
    outputs = fn(self.layer1(x))
    outputs = self.layer2(outputs)
```

Figure 13: Part G, Forward

viii. **(2 pts)** Create an instance of `TwoLayerNetwork`, set up a criterion with `torch.nn.CrossEntropyLoss`, and set up a SGD optimizer with learning rate 0.0005 by using `torch.optim.SGD`. Then train `TwoLayerNetwork`.

Solution:

```
### part h: prepare TwoLayerNetwork, criterion, and optimizer
model_two = TwoLayerNetwork()
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(params=model_two.parameters(), lr=0.0005)
```

Figure 14: Part H

```
Start training TwoLayerNetwork...
| epoch  1 | train loss 1.112896 | train acc 0.333333 | valid loss 1.1135
| epoch  2 | train loss 1.104948 | train acc 0.333333 | valid loss 1.1055
| epoch  3 | train loss 1.100852 | train acc 0.333333 | valid loss 1.1014
| epoch  4 | train loss 1.098154 | train acc 0.330000 | valid loss 1.0987
```

Figure 15: Results Two Layer

Performance Comparison [16 pts]

ix. **(3 pts)** Generate a plot depicting how `one_train_loss`, `one_valid_loss`, `two_train_loss`, `two_valid_loss` varies with epochs. Include the plot in the report and describe your findings.

Solution: Overall, there was more variation in loss across epochs for model one than model two. Model two consistently had higher loss in both train and validation data than model one. Both of the model's losses got lowest at epoch 30.

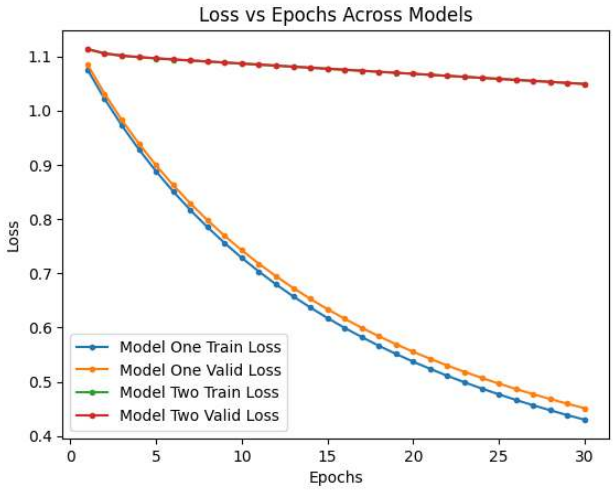


Figure 16: Loss vs Epoch, SGD

x. **(3 pts)** Generate a plot depicting how `one_train_acc`, `one_valid_acc`, `two_train_acc`, `two_valid_acc` varies with epochs. Include the plot in the report and describe your findings.

Solution: Overall, model one consistently maintained higher train and validation set accuracy than model two. However, model two consistently made increasing progress and almost approached model's ones accuracy by epoch 30, although it took more epochs to get to that accuracy than the other model did.

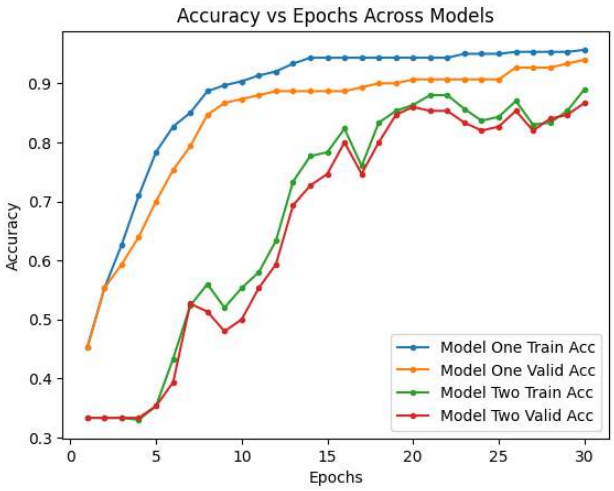


Figure 17: Accuracy vs Epochs, SGD

xi. **(3 pts)** Calculate and report the test accuracy of both the one-layer network and the two-layer network. How can you improve the performance of the two-layer network ?

Solution: We can improve the performance of the two-layer network by using a different optimizer that might be better suited. We can also increase the number of hidden dimensions, which is currently 400.

Model One Test Accuracy: 0.9599999785423279

Model Two Test Accuracy: 0.8933333158493042

xii. **(7 pts)** Replace the SGD optimizer with the Adam optimizer and do the experiments again. Show the loss figure, the accuracy figure, and the test accuracy. Include the figures in the report and describe your findings.

You can refer to https://pytorch.org/docs/stable/optim.html for more information about `torch.optim.Adam`.

Solution: Immediately, we see that the loss is lower across both models and all lines than it was with SGD. Although we start high, we get to a lower loss in much fewere epochs than we did with SGD. The difference in loss is also much closer between the two models than before. Adam seems to be a better fit for both models, since the lowest loss is lower than with SGD (0.0-0.2 vs 0.4 and 1.1). In terms of test accuracy, we see an increase for both models, although it is of a higher magnitude for model two. In terms of train and validation accuracy, model two is again much closer to model's one's results than was the case with SGD. Rather than taking 20 epochs to get close to model one's accuracies, it only took a little over 5 this time. The accuracy, on average, is higher than with SGD.

Model One Test Accuracy: 0.9666666388511658
Model Two Test Accuracy: 0.9599999785423279

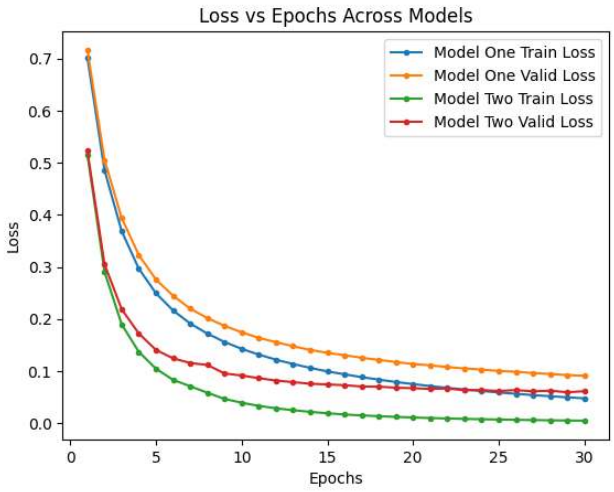


Figure 18: Loss vs Epochs, Adam

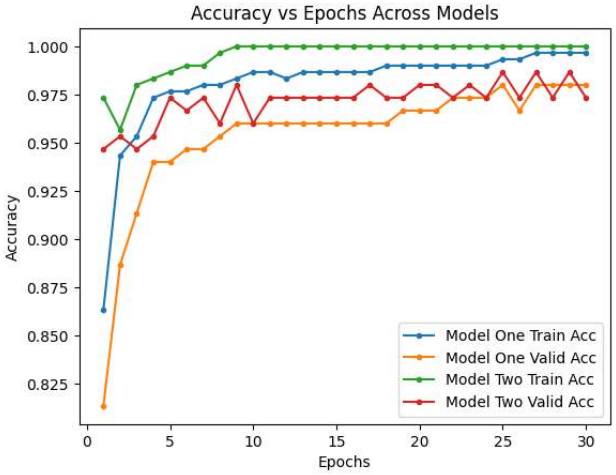


Figure 19: Accuracy vs Epochs, Adam